

Systementscheidungen

Software Engineering Gruppe 12:

Dominik Glozinic, Niklas Bachmann, Ahmad Al Mhazam

App & Sprache

Plattform

Entschieden: Handy, Android

- Die App soll von jedem und überall gespielt werden können. Eine Quiz-App soll außerdem kompakt und einfach bedienbar sein, weshalb wir uns für eine Handy-App entschieden haben. Eine PC-App wäre unpraktischer, und wir haben weder die Kapazitäten noch die Zeit, sie plattformübergreifend zu entwickeln.
- Wir hatten die Wahl zwischen Android und iOS. Android ist deutlich einfacher als iOS und wurde deshalb ausgewählt.

IDE

Entschieden: Android Studio

- Besser als die IntelliJ-Alternative für reine Android-Apps wegen eingebautem Emulator

Programmiersprache

Entschieden: Kotlin

- Alternative: Java
 - Kotlin ist eine modernisierte Version von Java mit vollständiger Abwärtskompatibilität – kein Nachteil beim Wechsel
-

UML-Diagramm

Tool: plantUML

Alternativen: Lucidchart, draw.io

(Beide waren webbasiert. Draw.io hatte nicht genug Funktionen und Lucidchart hatte die meisten Funktionen hinter einer Bezahlsschranke gesperrt.)

- Bei der Auswahl des Tools hatten wir 3 Kriterien:

1. Die Diagrammerstellung soll komfortabel und einfach sein
 2. Das Diagramm soll leicht veränderbar sein, ohne Zeit damit zu verschwenden, den Rest des Diagramms neu zu erstellen oder zu verschieben
 3. Das Diagramm wird später für die Struktur unserer App verwendet, und wir wollten ein Tool, das es ermöglicht, das Diagramm in ein Code-Skelett umzuwandeln
- plantUML ist ein open source, textbasierter Diagrammersteller. Der textbasierte Ansatz ist für uns ein großer Vorteil, da Klassen und Verbindungen einfach hinzugefügt und umgeschrieben werden können. Außerdem erstellt die automatische Text-zu-Grafik-Umwandlung in Sekunden ein übersichtliches und gut lesbares Diagramm.
 - Da plantUML textbasiert ist, kann es von KI und Erweiterungen leicht gelesen werden, was die spätere Übertragung des Codes in ein App-Skelett erleichtert

Aufteilung

Aufteilung in:

1. Darstellung
 2. Logik
 3. Netzwerk & Persistenz
- Darstellung ist unsere oberste Schicht, da sie die gesamte Benutzeroberfläche enthält, mit der der Nutzer interagiert. Wir haben uns dagegen entschieden, diese Schicht in unserem UML-Klassendiagramm zu modellieren, da sie für die Logik nicht wirklich relevant ist und das Diagramm nach unserem Ermessen unnötig verkomplizieren würde.
 - Die Logik-Schicht enthält den Großteil der App-Logik. Sie soll die Nutzer- und Quiz-Definitionen sowie die Host- und Spielerfunktionen (Spiel spielen und Quiz erstellen) beinhalten.
 - Netzwerk und Persistenz sind als Hilfschichten für die Logik-Schicht gedacht. Persistenz übernimmt die Quiz-Speicherung und Netzwerk die Kommunikation zwischen den Geräten.

Ein weiterer Grund für die 3 getrennten Schichten ist, dass unser Team aus 3 Personen besteht, was die Aufteilung der Arbeit erleichtert.

Logik

- Spieler und Host haben denselben Ursprung – Eine einfache und naheliegende Möglichkeit, die App in ihre 2 Modi (Host und Spieler) aufzuteilen. Dies unterteilt auch die Funktionen der App übersichtlich. Der Host kann ein Quiz erstellen und hosten, und der Spieler kann beitreten.
- Answer Question und Quiz Session sind per Aggregation verbunden

- Verwendung von Schnittstellen zur Verbindung der 3 Schichten -> Dies war die beste Möglichkeit, die verschiedenen Schichten zu verbinden und klare Grenzen zu schaffen, damit nicht 2 Personen am selben Code arbeiten.
 - Da wir Git verwenden, ist dies für uns besonders wichtig, weil wir die Situation vermeiden wollen, in der 2 Personen an derselben Klasse arbeiten und ihre Arbeit beim Committen zerstören.
 - Ursprünglich hatten wir die Schnittstellen weggelassen, aber die Klassen, in die sie eingehen, wären zu groß geworden und die oben genannte Situation wäre unvermeidbar gewesen.
- Eine wichtige Entscheidung war, wo die Punkte berechnet werden. Wir hatten zwei Möglichkeiten:
 1. Die richtige Antwort wird an den Host gesendet und die Spielerpunkte dort berechnet
 2. Die richtige Antwort und die Punkteberechnung befinden sich auf dem Gerät des Spielers

Option 2 wäre deutlich einfacher zu implementieren, aber wir haben uns letztendlich für Option 1 entschieden, da Option 2 ein wesentliches Problem hat: Die Punkte würden nur davon abhängen, wann der Spieler geantwortet hat, nicht aber an welcher Position er geantwortet hat. Ein weiterer Vorteil von Option 1 ist, dass es sicherer ist, da das Punktesystem manipulationsresistent sein soll, um Schummeln zu verhindern.

Aufgrund der erhöhten Komplexität mussten wir eine zweite Answer-Klasse hinzufügen, die die Antwort des Spielers enthält.

Netzwerk

Lokaler Server

1. SSH – zu komplex
2. HTTP – erlaubt mehrere gleichzeitige Verbindungen (entschieden)

Welcher HTTP-Server?

1. Ktor – Standard-HTTP-Framework für Kotlin
2. Netty – sehr performant
3. NanoHTTPD – leichtgewichtig, fertig implementierter Server (entschieden)

Begründung

Obwohl NanoHTTPD für Java konzipiert ist, ist es am einfachsten einzusetzen und zu integrieren. Netty ist deutlich performanter und Ktor besser für Kotlin geeignet – ihre Komplexität ist für dieses Projekt jedoch nicht notwendig.

Netzwerkerkennung

1. Automatische Suche nach lokalen Quizen
2. **Manueller Beitritt per Raumcode (entschieden)**

Begründung

Schwierig zuverlässig umzusetzen, besonders in großen Netzwerken mit Geräten ohne die App. Ein Einladungssystem (Master sendet Anfrage an alle Geräte) erzeugt unnötigen Datenverkehr, kann lange dauern (z.B. in Schulnetzwerken) und könnte von einer Firewall blockiert werden.

Lösung für den Raumcode:

Der manuelle Beitritt ist umständlicher, aber deutlich einfacher. Als Abhilfe: Serveradresse als QR-Code komprimieren, den Nutzer scannen lassen. Googles ML Kit Barcode Scanning API bietet eine einfach zu integrierende Lösung.

IHostNetwork und IPlayerNetwork

IHostNetwork und IPlayerNetwork eignen sich als Schnittstellen zwischen Logik und Netzwerk, sodass die oben genannte Option vermieden wird.

Persistenz

Speichermedium: SQLite

- Ursprünglich hatten wir geplant, die Quizzes in einer Datei zu speichern. Das hätte sich jedoch aufgrund der Lese- und Schreibrechte von Android als schwierig erweisen können. SQLite ist eine hervorragende Option, da es leichtgewichtig ist, fast keine Konfiguration erfordert (es wird kein Hintergrund-Server benötigt) und direkt im Android-Betriebssystem integriert ist. Zudem setzt es keine Internetverbindung voraus (im Gegensatz zu traditionellen SQL-Datenbanken wie MySQL). Android Studio verfügt über einen integrierten Database Inspector, mit dem sich Änderungen in SQL-Datenbanken in Echtzeit visuell verfolgen lassen. Das macht die Implementierung von SQL wesentlich einfacher als ein eigenes, dateibasiertes Speichersystem.

QuizRepositoryImpl

- Eine Herausforderung bei unserem SQL-Ansatz ist, dass wir für die Antworten das JSON-Datenformat nutzen. Daher benötigten wir eine Konverter-Klasse (TypeConverter), die die Kotlin-Objekte in JSON-Daten für die Speicherung übersetzt und umgekehrt.

- JSON ist auch der Grund, warum wir auf der Datenbank-Ebene keine separate Klasse für die Antworten (Answer) haben. Bei der Übersetzung werden die Frage und die dazugehörigen Antworten in ein einziges Objekt komprimiert. Das ist einfacher zu speichern als vier separate Objekte, die später wieder zusammengeführt werden müssten (QuestionEntity).
- Die QuizEntity und die QuestionEntity werden getrennt voneinander gespeichert. Dies hat folgende Gründe:
 1. Vermeidung eines One-to-Many-Problems. Dass eine Frage 4 Antworten beinhaltet, ist unproblematisch. Wir waren uns jedoch unsicher, ob ein Quiz mit beispielsweise 30 Fragen als ein einziges Objekt fehlerfrei gespeichert werden kann.
 2. Dieses Format ermöglicht es uns, das Quiz Frage für Frage nachzuladen, anstatt das gesamte Quiz im Voraus komplett in den Speicher laden zu müssen